

How do you determine whether a test was successful or not? You would probably say that if no defects are found, then the test is successful—and if some defects are found, then the test is *unsuccessful*. This is completely contrary to the idea behind testing!

To understand it better, let us take a simple analogy. Suppose a patient is very sick and is suffering from a high fever. He visits the doctor for a diagnosis. The doctor recommends some tests, but they don't detect any problem. So, we say that the tests were unsuccessful in detecting the problem. However, if the tests were able to detect the problem, the doctor can begin treatment of the patient—that's a *successful* test. The same is true for software testing. Assume that your program is like a sick patient. Now, it is up to you to design the tests to detect the problems. So, one can define a successful test as one that is able to detect defects, and an unsuccessful test as one that can't find any.

As should be evident by now, it is all about having the right attitude to testing. As is rightly said, "You will see what you want to see." If your intention is to show that your program is defect-free, for sure, you are unlikely to find any defects.

Test cases

What is a test case, and what should it contain? In simple words, a test case is a set of variables or conditions that are used to test a program. A typical test case is expected to contain the following information:

- Test case ID
- Description of the test case
- Expected input
- Expected output
- Actual result (PASS / FAIL)
- Configuration (hardware/software)
- Steps to run the test case
- Author
- Date

Recalling what we've discussed earlier, a good test case is one that has a high probability of finding defects.

So how do you write test cases? That depends on how large and complex the program or functionality to be tested is. If the program/functionality is too small (say, a program with 8-10 lines of code), it might make sense to write test cases manually. However, if the program is big, use scripting languages to write the test cases. For example, the open source CPython, one of the most widely used implementations of the Python language, can be used to write test scripts. Other popular scripting languages include JavaScript, Perl, PHP and Ruby.

Now, once you have written the test cases, how would you ensure that you have enough? How many test cases would you need to write to ensure that there are no defects?

It has been often observed (and my own experience corroborates this finding), that people write tests only as long as they can find defects. Most of them get tired or bored if they keep testing and no defects are found. In such cases, developers assume that their code works fine, and keep on adding new code without any tests. I am sure most project managers would agree with this observation.

If you aren't able to produce any effective results in testing, you probably need to re-check your testing process and approach. Often, the testing involves adding a large number of tests, but the tests are written randomly, and the testing tends to become aimless.

While testing, it is extremely important to clearly classify the areas to be tested. You need to check not only the positive patterns but the negative patterns as well. (I will share more details on this in my next article.) It is also very important to check boundary conditions.

As I mentioned earlier, you must not only check for whether the program is working correctly, but also check whether it is doing something it ought not to—there might be some hidden behaviour that you might not be aware of, and which might only get uncovered after the product has been shipped to the customer. This reminds me of an interesting article by S G Ganesh in the *LFY* April 2012 issue: "A Bug or a Feature?". As he rightly puts it, "... the customers don't know that it is a bug, but think it is a feature, and start using it. Now, even when you realise that it is a bug, you cannot fix it." So, if you ship the product with a bug, and the customer gets used to it as a feature, you will not be able to undo it, and will have to support the bug in future releases as well.

Most of the time, developers test in and around their code, keeping the design in mind. However, rather than the design, it is important to keep the customer or the end-user in mind, while testing. After all, they are the ones who will be using the product, not you!

In my next article, I will explain the various kinds of testing techniques that are available. Till then, here are some suggestions for further reading. 

References and suggested reading

- [1] http://en.wikipedia.org/wiki/Scripting_language
- [2] Article by S G Ganesh in the April 2012 issue of *LFY*, 'A Bug or a Feature?'
- [3] *The Art of Software Testing*, by Glenford J Myers, Tom Badgett, Todd M Thomas

By: Deepti Sharma

The author is an open source enthusiast with almost a decade of experience in the design, development and quality validation of systems software tools (compilers, assemblers and device drivers) for embedded systems. She loves writing for OSFY. She can be reached at deepti.acmet@gmail.com.